	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 16
	Name: KOLLURU NAGA SINDHU Reg. No: 2503A52017
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 06-10-2026

TASK 1 – Improve Readability

1. **ORIGINAL CODE**

```
a = 80
b = 70
c = 90

x = (a + b + c) / 3

if x >= 50:
   print("pass")
else:
   print("fail")
```

2.PROMPT

Refactor the following Python code to improve readability.

Requirements:

1. Use meaningful variable names instead of a, b, c, and x.
2. Create a function named calculate_average().
3. Improve formatting and readability.
4. Keep the original behavior and output unchanged.
5. Explain the improvements made.

Legacy code:

```
a = 80
b = 70
c = 90

x = (a + b + c) / 3

if x >= 50:
print("pass")
else:
print("fail")
```

3.AI Suggestion

Use meaningful variable names such as mark1, mark2, and mark3.

Replace x with average.

Create a reusable calculate_average() function.

Improve indentation and spacing.

Keep the same pass/fail condition.

4. Refactored Code

```
def calculate_average(mark1, mark2, mark3):
    return (mark1 + mark2 + mark3) / 3


mark1 = 80
mark2 = 70
mark3 = 90

average = calculate_average(mark1, mark2, mark3)

if average >= 50:
    print("pass")
else:
    print("fail")
```

6. **Test Output**
   Original Code output:

```
pass
```

Refactored Code output:

```
pass
```

Test Result: Both versions produced the same output.

7. **Explanation**

The code was improved by replacing unclear variable names with meaningful names such as mark1, mark2, mark3, and average. A calculate_average() function was created to make the calculation reusable. Proper formatting and indentation were also added to improve readability. The original and refactored versions produced the same output, so the original behavior was preserved.

Ta sk 2 – Remove Repeated Code

1. Original Code

name1 = "Arun"

marks1 = 85

```
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 = "Priya"
marks2 = 72
```

2. **Prompt**

   Identify the duplicated logic in the following Python code and refactor it.

   Requirements:

1. Identify the repeated logic.
2. Create a reusable function to handle the pass/fail checking.
3. Test the function with at least four students.
4. Keep the output behavior correct.
5. Explain why removing duplicated code improves maintainability.

   Legacy code:

   name1 = "Arun"
   marks1 = 85
   if marks1 >= 50:
   print(name1, "Pass")
   else:
   print(name1, "Fail")
   name2 = "Priya"
   marks2 = 72
   if marks2 >= 50:
   print(name2, "Pass")
   else:
   print(name2, "Fail")

3. AI Suggestion

- The if-else pass/fail logic is repeated for every student.
- Create a reusable function called check_result().
- Pass the student's name and marks to the function.
- Use the same function for all four students.
- This reduces duplicate code and makes future changes easier.

4.Refactored Code and output

```
def check_result(name, marks):
    if marks >= 50:
        print(name, "Pass")
    else:
        print(name, "Fail")


check_result("Arun", 85)
check_result("Priya", 72)
check_result("Rahul", 45)
check_result("Meena", 60)
```

```
Arun Pass
Priya Pass
Rahul Fail
Meena Pass
```

Test Result: The function was tested with four students, including both passing and failing marks.

5.Explanation

The duplicated pass/fail logic was identified and moved into a reusable check_result() function. The function takes the student's name and marks as input and determines whether the student passed or failed. It was tested with four students and produced the correct results. Removing duplicated code improves maintainability because if the pass/fail rule needs to be changed later, it only needs to be changed in one place.

Task 3 – Refactor a Long Conditional

1. **Original Code**

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

2. **Prompt**

   Refactor the following Python code to simplify the long conditional logic.

   Requirements:

1. Simplify the conditional logic without changing its behavior.
2. Create a function named determine_priority().
3. Test the function with at least four different age and temperature combinations.
4. Compare the original and refactored logic.
5. Explain whether the refactored code preserves the original behavior.

   Legacy code:

```
age = 68
temperature = 39
if age >= 60 and temperature >= 38:
print("High Priority")
elif age >= 60 and temperature < 38:
print("Medium Priority")
elif age < 60 and temperature >= 38:
print("Medium Priority")
else:
print("Normal Priority")
```

3. AI Suggestion

- The first condition is the only situation that gives **High Priority**.
- If the patient is both **60 or older** and has a temperature of **38 or higher**, the priority is high.
- All other cases except the final normal case can be simplified.
- Create a determine_priority() function to make the logic reusable.
- The refactored code should preserve the original behavior.

4. Refactored Code & Output

Test Result: All four age/temperature combinations produced the expected resu

```python
def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"


# Test 1
print(determine_priority(68, 39))

# Test 2
print(determine_priority(65, 37))

# Test 3
print(determine_priority(45, 39))

# Test 4
print(determine_priority(40, 37))
```

```
High Priority
Medium Priority
Medium Priority
```

Test Result: All four age/temperature combinations produced the expected results.

6. Explanation

The long conditional logic was simplified by using an or condition for the Medium Priority cases. A determine_priority() function was created so the logic can be reused easily. Four different age and temperature combinations were tested. The refactored code produces the same results as the original logic, so the original behavior is preserved while the code is shorter and easier to understand.

Task 4 – Refactor Inefficient List Processing

1. Original Code

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412


found = False


for patient_id in patient_ids:
    if patient_id == target:
        found = True


if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

2. Prompt

Refactor the following Python code to improve readability and efficiency.

Requirements:

1. Improve the readability of the code.
2. Determine whether a set or dictionary is better for frequent patient ID lookups.
3. Implement the justified suggestion.
4. Test the code with an existing patient ID and a missing patient ID.
5. Compare the typical lookup complexity of a list and a set.
6. Explain why the chosen data structure is better.

Legacy code:

```
patient_ids = [101, 205, 309, 412, 518, 623]
target = 412

found = False

for patient_id in patient_ids:
if patient_id == target:
found = True

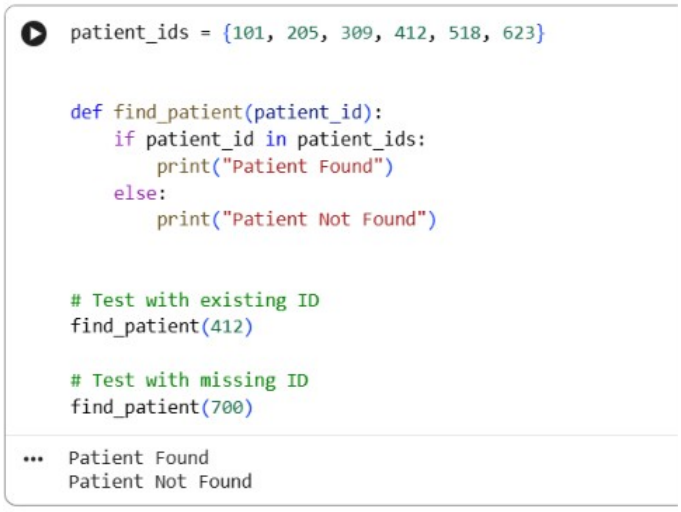
if found:
print("Patient Found")
else:
print("Patient Not Found")
```

3. AI Suggestion

- The original code manually loops through the list to find the patient ID.
- A **set** is better for frequent lookups because it is designed for fast membership checking.
- Use the in operator instead of writing a manual for loop.
- A set lookup is typically **O(1)** on average, while a list lookup is **O(n)**.

- A dictionary would be useful if we also needed information associated with each patient ID, but for checking whether an ID exists, a **set** is sufficient.

4. Refactored Code & Output



```
patient_ids = {101, 205, 309, 412, 518, 623}


def find_patient(patient_id):
    if patient_id in patient_ids:
        print("Patient Found")
    else:
        print("Patient Not Found")


# Test with existing ID
find_patient(412)

# Test with missing ID
find_patient(700)
```

```
Patient Found
Patient Not Found
```

6. Explanation

The original code manually loops through the list to find a patient ID. The code was improved by using a set because the program only needs to check whether an ID exists. The in operator makes the code shorter and easier to read. A set provides O(1) average-time lookup, while a list generally requires O(n) time. The refactored code was tested with both an existing and a missing patient ID and produced the correct results.

Task 5 – Complete Legacy Code Review with AI

1. Original Code

students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0

```
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

2. Prompt

Review and refactor the following legacy Python code.

Requirements:

1. Identify at least five improvements.
2. Use meaningful variable names.
3. Create reusable functions where appropriate.
4. Use suitable Python features when they improve clarity.
5. Run both the original and refactored versions and verify equivalent results.
6. Record which AI suggestions were accepted or rejected and give reasons.
7. Identify the most useful refactoring change and justify it.

Legacy code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
total = total + s[1]

avg = total / len(students)

for s in students:
if s[1] >= 50:
print(s[0], "Pass")
else:
print(s[0], "Fail")

print("Average =", avg)
```

3. AI Suggestion

The following improvements were suggested:

1. Use meaningful names such as student_records, student, marks, and average_marks.
2. Create a calculate_average() function.
3. Create a display_results() function for the pass/fail logic.
4. Use sum() with a generator expression to calculate the total marks clearly.
5. Use tuple unpacking (name, marks) instead of repeatedly using indexes such as s[0] and s[1].

4. Refactored Code

```
student_records = [
    ("Arun", 78),
    ("Priya", 45),
    ("Rahul", 88),
    ("Meena", 52)
]


def calculate_average(records):
    total_marks = sum(marks for name, marks in records)
    return total_marks / len(records)


def display_results(records):
    for name, marks in records:
        if marks >= 50:
            print(name, "Pass")
        else:
            print(name, "Fail")


display_results(student_records)

average_marks = calculate_average(student_records)
print("Average =", average_marks)
```

5. Test Output

Original Code output:

```
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
```

Refactored Code output:

```
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
```

Test Result: Both versions produced the same output, so the refactored code preserves the original behavior.

6. Explanation

The legacy code was improved in several ways. Meaningful variable names were used instead of short names such as s and avg. The average calculation was moved into a reusable calculate_average() function, and the pass/fail logic was moved into display_results(). The sum() function and tuple unpacking were also used to make the code clearer. All five suggested improvements were accepted because they improve readability and maintainability without changing the program's result.

The most useful refactoring change is creating reusable functions because each function has one clear purpose. This makes the program easier to understand, test, maintain, and reuse. The original and refactored programs produced the same output, confirming that the behavior was preserved.